



DATA STRUCTURES

CONTENTS



32. Introduction to Graphs

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. In this module we will discuss the fundamentals of another important data structure –the graphs.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the components of Graphs
- To describe the Graph ADT
- To discuss the different representations of Graphs
- To analyse the different representations of Graphs

32.1 Introduction

Graph is a data structure that consists of a set of vertices (nodes) and a set of

edges between the vertices. The set of edges describes relationships among vertices. Graphs are similar to trees except they do not have as many restrictions.

- Graphs represent relationships among data items.
- A **graph** $G = (V, E)$ consists a set of **vertices**, V , and a set of **edges**, E .
- Each edge is a pair of vertices (v, w) , where both v, w belong to V
- A sub graph consists of a non-empty subset of a graph's vertices V and a subset of (possibly empty) of its **edges** E where each edge is between vertices that form a subset of V with cardinality 2 (an unordered pair).

A graph with six vertices and seven edges is shown in Figure 32.1.

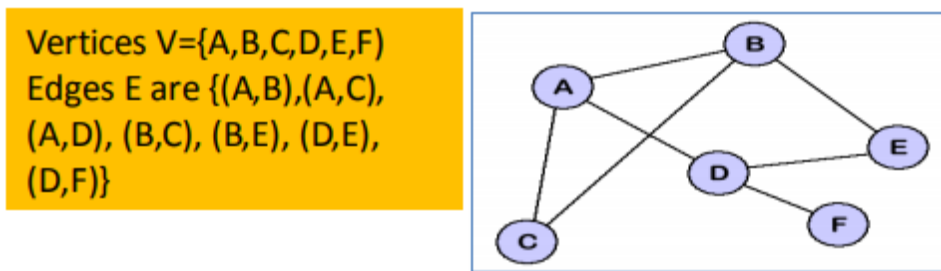


Figure 32.1 Example of a graph

32.2 Terminology

A graph is called a **dense** graph when $|E| \gg |V|^2$ and a **sparse** graph when $|E| \ll |V|$. An **undirected graph** is a graph where the pair of vertices are unordered and any Edge $(u, v) = \text{Edge}(v, u)$. A graph is called a **directed graph** when each edge connects two vertices, called the source (u) and destination (v); the edge connects the source to the destination (the order is significant). A directed edge (u, v) goes from vertex u to vertex v , notated $u \rightarrow v$. A **weighted graph** associates weights with either the edges or the vertices. The **complete graph** is a graph in which every vertex is directly connected to every other vertex. A complete directed graph is shown in Fig 32.2. The number of edges in a complete directed graph with N vertices is given by $N * (N-1)$ is of the order of $O(N^2)$ (Figure 32.2). A path is a sequence of vertices that connect two nodes in a graph. The **length of a path** is the number of edges on it. A Path is called **simple** if vertices in the sequence are distinct.

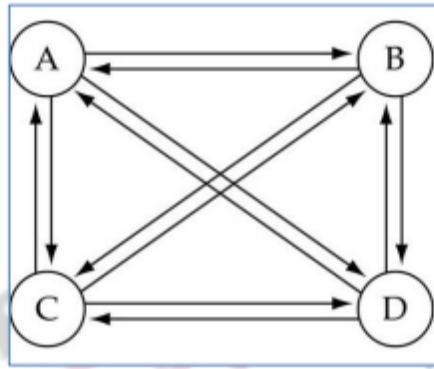


Figure 32.2 Example of a Complete Directed Graph

Two nodes of an undirected graph are **called adjacent nodes** if they are connected by an edge. If (v_0, v_1) is an edge in an undirected graph, then vertices v_0 and v_1 are said to be adjacent. The edge (v_0, v_1) is incident on vertices v_0 and v_1 . If $\langle v_0, v_1 \rangle$ is an edge in a directed graph where v_0 is the source node and v_1 is the destination node then v_0 is said to be **adjacent to v_1** , and v_1 is said to be **adjacent from v_0** . The edge $\langle v_0, v_1 \rangle$ is incident on v_0 and v_1 . A **forest** is an acyclic graph, and a **tree** is a connected acyclic graph. A **subgraph** of G is a graph G' such that vertices $V(G')$ is a subset of vertices $V(G)$ and edges $E(G')$ is a subset of edges $E(G)$ between vertices that are from the subset $V(G')$. In other words a subgraph of G is a graph G' such that $V(G') \subseteq V(G)$ and $E(G') \subseteq E(G)$.

Examples of subgraphs are shown in Fig 32.3

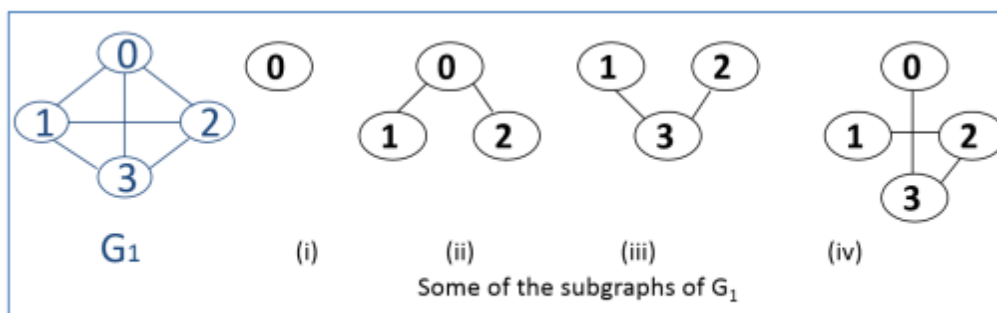


Figure 32.3 Sub-graphs of Graph G_1

32.2.1 Connected Component

In an undirected graph G , two vertices, v_0 and v_1 , are connected if there is a path in G from v_0 to v_1 . An undirected graph is connected if, for every pair of distinct vertices v_i, v_j , there is a path from v_i to v_j . A connected component of an undirected graph is a maximal sub-graph that is connected. A completely connected graph has exactly 1 component. Example of connected components and not a component are shown in Fig 32.4.

A directed graph is strongly connected if there is a directed path from v_i to v_j and also from v_j to v_i . A strongly connected component is a maximal subgraph that is strongly connected.

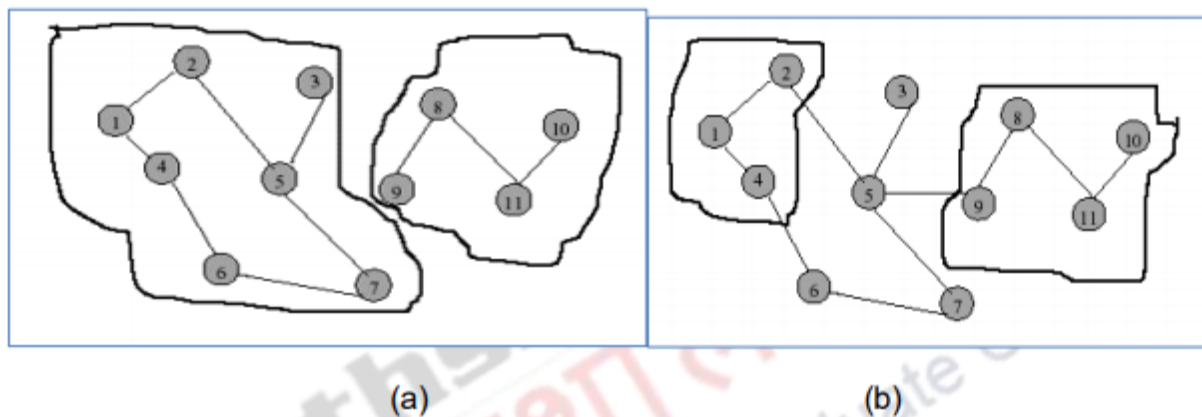


Figure 32.4 (a) Connected Components and (b) Not Components

32.2.2 Degree

The degree of a vertex in an undirected graph is the number of edges incident to that vertex. If d_i is the degree of a vertex i in an undirected graph G with n vertices and e edges, the number of edges is

$$e = (\sum_0^{n-1} d_i) / 2$$

Thus for a complete undirected graph, the number of edges = $n(n-1)/2$ edges

Graphs with different degree are shown in Fig 32.5

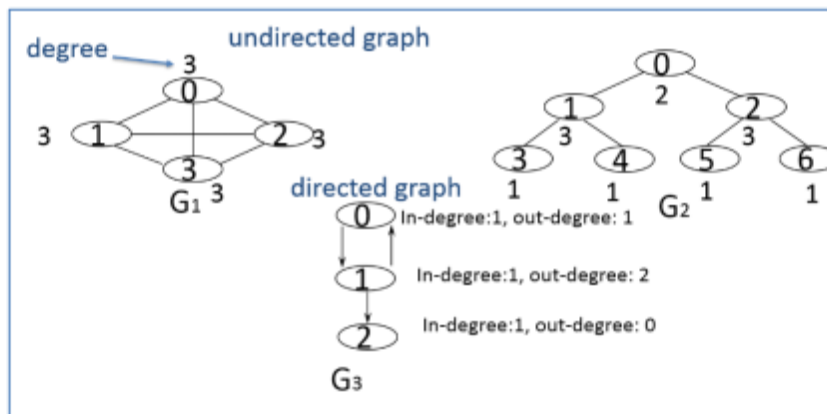


Figure 32.5 Examples for Degree of Graphs

For a directed graph, the in-degree of a vertex v is the number of incoming edges that have v as the head and the out-degree of a vertex v is the number of outgoing edges that have v as the tail. For a complete directed graph the number of edges = $n(n-1)$ edges

32.3 Trees as Graphs

Every tree is a connected acyclic graph that is a graph with some restrictions as shown in Fig 32.6. The restrictions are that the tree is rooted, *directed*, there are *no* cycles and there is a *directed path* from the root to every node.

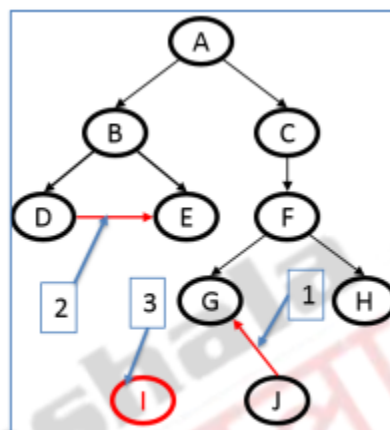


Figure 32.6 Example of Tree as Graph

Each of the red areas breaks one of these constraints. In the tree the direction of the edge is assumed to be from a node to another node that is at a lower level though the direction is not explicitly shown. In the figure 32.6, 1 shows the direction is from lower level node to higher level and is not allowed in a tree. 2 shows the presence of a cycle while 3 violates the condition that there must be a directed path from the root to every node.

32.4 Why Use Graphs?

Graphs can be used to model a wide range of applications. When we design the model we need to decide on what components of the application represent the vertices and what connections between these components represent the edges. Examples of such applications include the following:

- Intersections and streets within a city
- Roads/trains/airline routes connecting cities/countries
- Computer networks
- Electronic circuits
- A layout of an adventure game world
- A schematic of the computers and connections that make up the Internet
- The links between pages on the Web
- The relationship between students and courses
- A diagram of the flow capacities in a communications or transportation network

32.4.1 Application1 –Air Flight System (Fig 32.8)

In this application, each vertex of the graph represents a city and each edge represents a direct flight between two cities. In this case a query on direct flights is equivalent to a query on whether an edge exists. A query on how to get to a location is equivalent to finding whether a path exists from A to B. We can even associate costs to edges (weighted graphs), then ask “what is the cheapest path from A to B” .

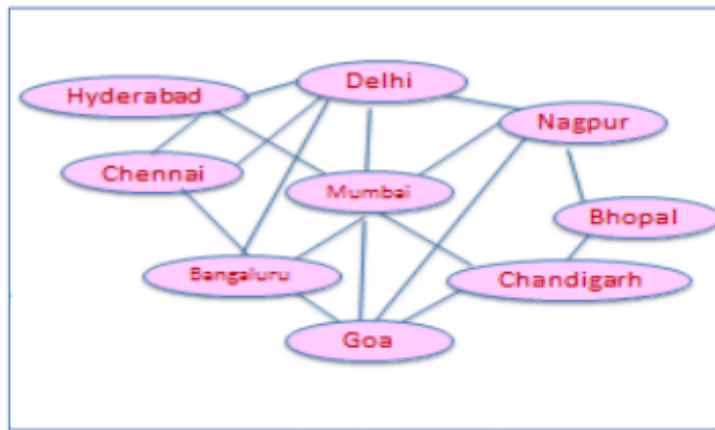


Figure 32.8 Air Flight System

32.4.2 Application2 –Wireless Communication

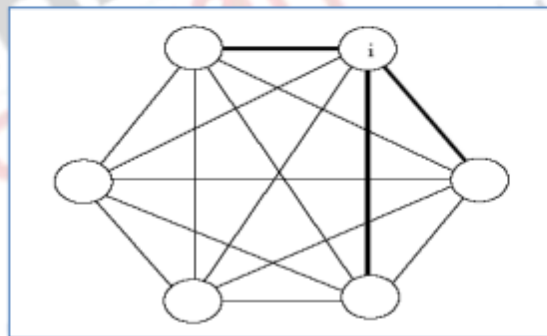


Figure 32.9 Wireless Communication

This application can be represented by a weighted complete graph (every two vertices are connected by an edge) (Figure 32.9). Each path represents the Euclidean distance between two stations. Each station uses a certain power i to transmit messages. Given this power i , only a few nodes can be reached (bold edges). A station reachable by i then uses its own power to relay the message to other stations not reachable by i . A typical wireless communication problem is: how to broadcast between *all* stations such that they are all connected and the power consumption is minimized. This is essentially the shortest path finding problem.

32.5 Representation of Graphs

Three popular computer representations of a graph represent the vertex set and the edge set, but in different ways.

32.5.1 Adjacency Matrix

This representation uses a 2D matrix to represent the graph. That is it is represented as a square grid of Boolean values. Therefore if the graph contains N vertices, then the grid contains N rows and N columns. For two vertices numbered I and J , the element at row I and column J is true if there is an edge from I to J , otherwise it is false.

The Matrix of will be of size $|V| \times |V|$

- One row and one column of the grid or matrix for each vertex In the case of a directed graph
- a cell in the matrix (row i and column j) contains a 1 if an edge exists from i to j , 0 if the edge does not exist.
- two cells in the matrix (row i and column j and row j and column i) both will contains a 1 if an edge between i and j , 0 if the edge does not exist.

The adjacency matrix for a directed graph is shown in Figure 32.10. **Space requirements:** For a graph with n nodes, adjacency matrices take $\Theta(n^2)$ space.

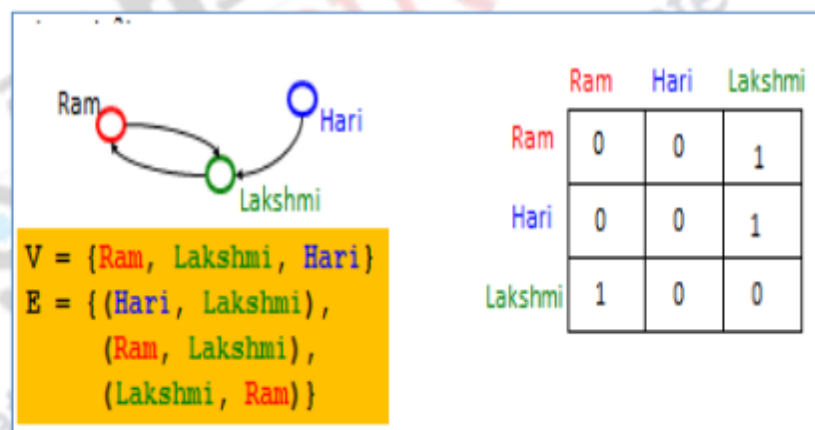


Figure 32.10 Adjacency Matrix

Some more examples for Adjacency Matrix for directed and undirected graphs are shown in Fig 32.11.

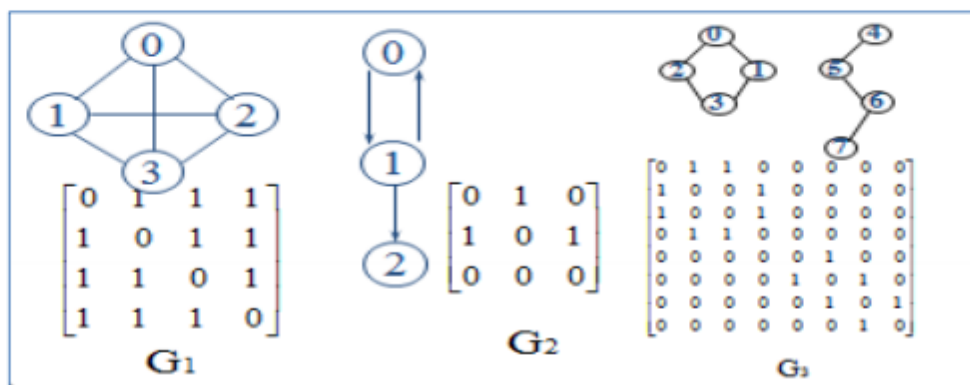


Figure 32.11 Adjacency Matrix -Examples

The adjacency matrix representation of a weighted graph is now discussed. The edges of a graph have weights assigned to them. These weights may represent the distance from one vertex to another or the cost of going from one vertex to another adjacent vertex. In the case of a weighted graph, matrix can contain the weight instead of 1. Now the adjacency matrix: $adj_mat[i][j]$ would keep the weights. Now we add a *weight* field to the node structure. A graph with weighted edges is called a *network*.

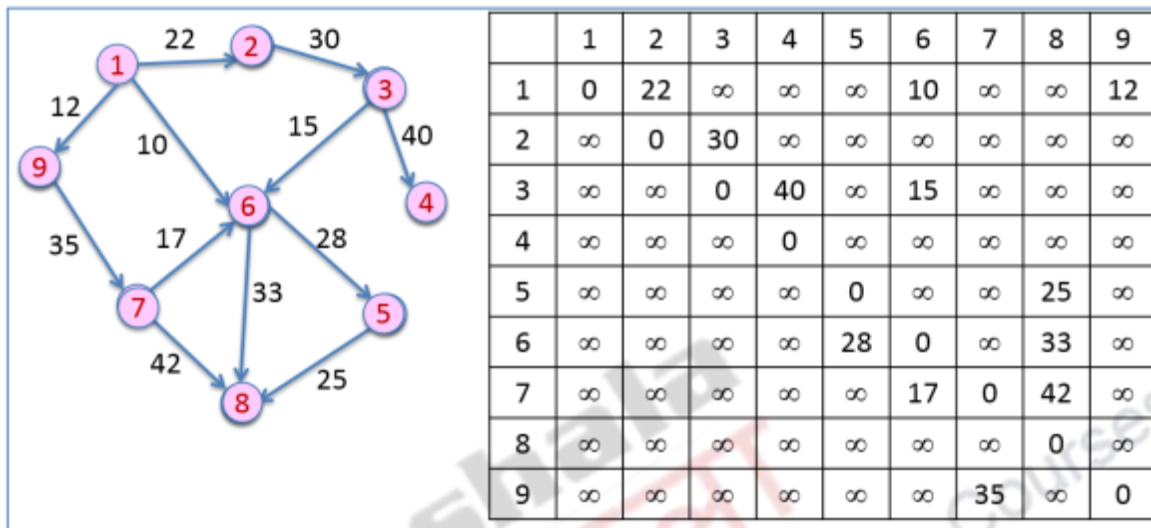


Figure 32.12 Adjacency Matrix of weighted graph

For an undirected graph, the degree of any vertex, i , is its row sum. For a directed graph, the row sum is the out-degree, while the column sum is the in-degree. The time complexity of checking edge number or examining if G is a connected graph is of the order of $O(n^2/2)$ if G is undirected and of the order of $O(n^2)$ if G is directed.

32.5.2 Adjacency List

In this representation we use a 1D array of linked lists. There is one list for each vertex in G . The nodes in list i represent the vertices that are adjacent to vertex i . For an undirected graph with n vertices and e edges, this representation requires n head nodes and n lists and $2e$ list nodes. A $|V|$ -ary list (array) in which each entry stores a list (linked list) of all adjacent vertices. The *adjacency list* representation of a graph $G = (V, E)$ consists of an array $Adj[1..|V|]$ of lists. Each list $Adj[v]$ is a list of all vertices adjacent to v . Figure 32.13 shows the adjacency list for a simple directed graph. Figures 32.14 shows the adjacency list for an undirected graph while Figure 32.15 shows the adjacency list representation for the same graph but now directed.



Figure 32.13 Adjacency List

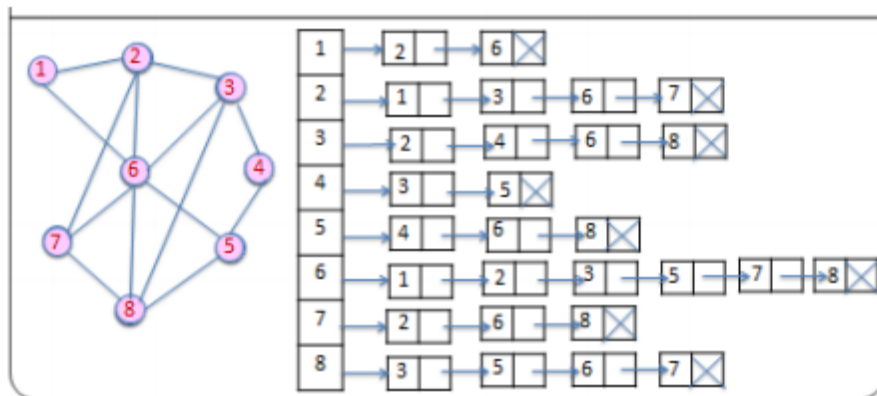


Figure 32.14 Adjacency List for an Undirected Graph

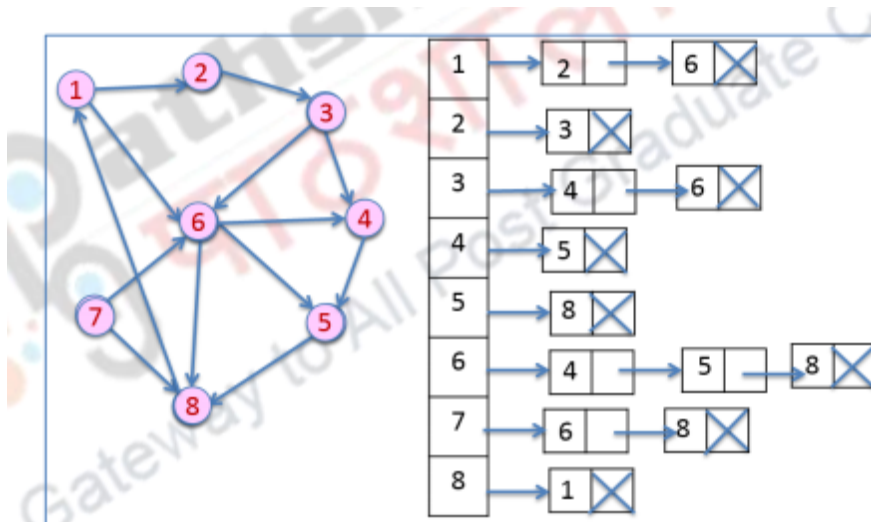


Figure 32.15 Adjacency List for a Directed Graph

32.5.2.1 Adjacency List- Operations

The following are the graph operations associated with the adjacency list

representation. The degree of a vertex in an undirected graph is the number of nodes in the adjacency list. The number of edges in a graph is determined in time of the order $O(n+e)$. The out-degree of a vertex in a directed graph is the number of nodes in its adjacency list. However to find the in-degree of a vertex in a directed graph we need to traverse the whole data structure.

32.5.2.2 Sequential Representation of Adjacency Lists

We will now discuss the sequential representation of adjacency lists. Here we sequentially pack the nodes on the adjacency lists (Fig 32.16). Here $node[1] \sim node[n+2e+1]$ may be used. The vertices adjacent from vertex i are stored in $node[node[i]], \dots, node[node[i+1]-1]$, $0 \leq i < n$. The number of edges in G may be determined in $O(n+e)$

- $node[0] \dots node[n-1]$: starting point for vertices
- $node[n]$: $n+2e+1$
- $node[n+1] \dots node[n+2e]$: head node of edge

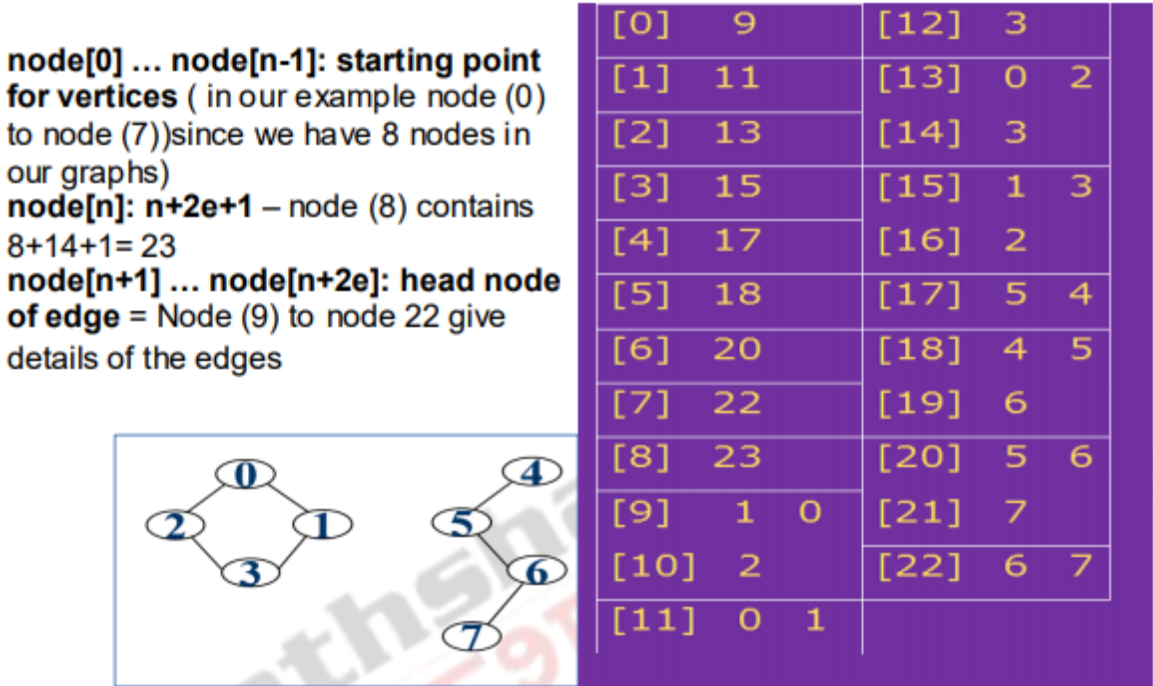


Figure 32.16 Sequential Representation of Adjacency List

Here we use the node 0 to n+2e i.e $8+2*7= 22$

Node 0 shows the location of the node where edges for vertex 0 starts (in our example 9), node 1 shows where the edges for 1 starts (in our example 11) and son on. In node 9 we have the edge with 0 (0-1) as edge while in node 10 we the next edge of 0 (0-2). Similarly in node 11 we have the edge with 1 (1 -0) as edge while in node 12 we the next edge of 1 (0-3) and so on.

32.5.3 Adjacency Multi-list

An edge in an undirected graph is represented by two nodes in adjacency list representation. Lists in which nodes may be shared among several lists that is a vertex shared by two different paths is called as adjacency multi-lists. There is exactly one node for each edge. This node is on the adjacency list for each of the two vertices it is incident to. Each node in the list is represented as given in Figure 32.17. An example of adjacency multi-list for an undirected graph is shown in Fig 32.18.

marked	vertex1	vertex2	path1	path2
--------	---------	---------	-------	-------

Figure 32.17 Node in an Adjacency Multi-list

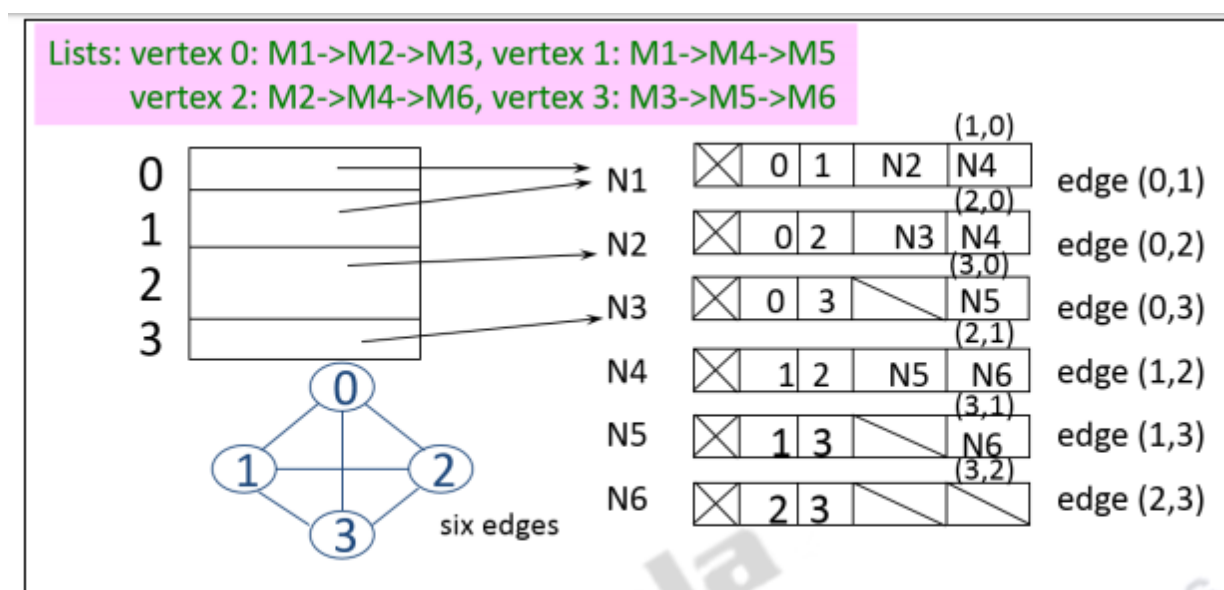


Figure 32.18 Adjacency Multi-List for an Undirected Graph

In Figure 32.18, the number of nodes is 4, the number of edges is 6. The Multi-list shows the structure where N1 shows the path with vertices 0 and 1 corresponding to edges (0-1) as well as (1-0) (since the example is an undirected graph). The first different path N2 containing first vertex 0 is shown next and the first different path N4 with vertex 1 is shown next. Next we go to N2 corresponding to path with vertices 0 and 2. Now N3 is the next path with 0 while N4 is the next path with 2. Next is N3 corresponding to path with vertices 0 and 3. Now there is no other path with 0 while N5 is the next path with 3. Next is N4 corresponding to path with vertices 1 and 2. Now N5 is the next path with 1 while N6 is the next path with 2. Next is N5 corresponding to path with vertices 1 and 3. Now there is no other path with 2 while N6 is the next path with 3. Finally we see N6 corresponding to path with vertices 2 and 3. Now we have processed all paths and there are no more paths to consider.

Analysis (Adjacency Matrix vs Adjacency List)

Now we will discuss the time complexity of graph operations when using adjacency matrix or adjacency list is used. For adding or removing edges in case of adjacency matrix the time needed will be a small constant but for adjacency list it will be of the order $O(N)$ where N is the number of vertices in the graph. Checking if an edge is present for the adjacency matrix representation time taken will be a small constant, however in the case of adjacency list the time taken is of the order $O(N)$ where the linked adjacency list has linear running time with the length of this list, on the average. Now for iterating through a vertex's edges the adjacency matrix takes time of the order of $O(N)$ while for the adjacency list it is of the order $O(E)$ where E is the number of edges in the graph. For finding all of the vertices adjacent to a given vertex, the adjacency list tends to support this operation more efficiently than the adjacency matrix. Both take of the order of $O(N)$ in worst case (i.e., when we are dealing with a complete graph). Now regarding the memory requirement, the adjacency matrix always requires N^2 cells while adjacency list requires an array of N pointers and a number of nodes equal to twice the number of edges in the case of an undirected graph.

32.5.4 Choice of Implementation

The choice which representation to use is normally based on which operations

are more frequent and whether a good set ADT is available. It also depends on the average number of edges per vertex, since the adjacency matrix is wasteful for sparse graphs, since we need $W(n^2)$ space.

Summary

- Explained the components of Graphs
- Described the Graph ADT
- Discussed the different representations of Graphs
- Analysed the different representations of Graphs

you can view video on Introduction to Graphs



Web Links
http://www.translationdirectory.com/glossaries/glossary272_d.php#9
http://www.csd.uwo.ca/courses/CS2210b/Lecture14.pdf
http://crypto.cs.mcgill.ca/~crepeau/CS250/2004/25.GraphsDS.pdf
Supporting & Reference Materials
https://www.cs.purdue.edu/homes/ayg/CS251/slides/chap9a.pdf
http://courses.cs.vt.edu/~cs3114/Fall09/wmcquain/Notes/T19.GraphStructures.pdf
http://www.cs.cmu.edu/afs/cs/academic/class/15210-f13/www/lectures/lecture09.pdf